

# Introduction to GitHub Copilot

Define, inspect, test, decide

Session 01 | 3 hours

# Agenda

---

| Segment                         | Time   |
|---------------------------------|--------|
| Proposal - and - evidence model | 7 min  |
| How context shapes a suggestion | 10 min |
| Access and data preflight       | 10 min |
| Inline suggestion controls      | 11 min |
| Prepared utility demonstration  | 12 min |
| Follow - up edits               | 6 min  |
| Fallback and lab handoff        | 4 min  |

# The durable workflow

---

```
graph TD; A[Define observable behavior] --> B[Check access, data, and scope]; B --> C[Inspect the proposed change]; C --> D[Run focused checks]; D --> E[Accept, revise, or reject];
```

**The developer owns the result.**

# Proposal is not proof

| Proposal               | Evidence                              |
|------------------------|---------------------------------------|
| Looks plausible        | A focused test passes                 |
| Uses familiar syntax   | The diff stays in scope               |
| Explains its reasoning | A reviewer can reproduce the result   |
| Matches one example    | Edge and failure behavior are checked |

Fluent output can still be incomplete or wrong.

# A practical system model

---

Task + approved context



Selected Copilot surface



Proposed output



Local checks + human review

Use this as a review model, not a fixed service diagram.

# Context changes the proposal

---

Useful context includes:

- function names and type hints;
- docstrings and examples;
- selected code and nearby files;
- tests and repository instructions.

More context is useful only when it is relevant and approved.

## Preflight before use

---

1. Confirm the account and surface.
2. Confirm the repository and data classification.
3. Limit the task to named files.
4. Name any metered-work owner and stop condition.
5. Prepare the manual route.

If approval is unclear, use the fallback.

## Access policy

---

The live lab requires GitHub Copilot in an approved editor.

The manual fallback uses the same:

- starter code;
- acceptance tests;
- review decisions;
- final verification.

**Access changes the tool path, not the learning objective.**

# Accept, revise, reject

---

**Accept** when the proposal meets the contract and fits the repository.

**Revise** when the direction is useful but a detail is wrong.

**Reject** when it changes scope, invents behavior, or is hard to verify.

Undo is a normal review action.

## Review one suggestion

---

Before accepting, ask:

- Which requirement does each line support?
- What happens on empty or unusual input?
- Did it add a dependency?
- Did it change a caller or public name?
- Which check will fail if the logic is wrong?

# Demo contract

---

```
def normalize_username(value: str) -> str:  
    """Trim outer whitespace, lowercase the value,  
    and join words with hyphens."""
```

```
assert normalize_username("  River Team  ") == "river-team"
```

The test makes repeated whitespace observable.

## Demo: inspect before accepting

---

Review the proposed implementation against three rules:

1. Outer whitespace disappears.
2. Case becomes lowercase.
3. Each run of internal whitespace becomes one hyphen.

Then run the focused test.

## A failing check is useful

---

This version looks reasonable but mishandles repeated spaces:

```
return value.strip().lower().replace(" ", "-")
```

The failure exposes a hidden assumption while the change is still small.

# Predicted follow-up edits still need review

---

Rename username to handle



Inspect each proposed caller update



Search for the old name



Run the focused test

A predicted edit can miss a reference.

# Manual fallback

---

When live access is unavailable:

1. Implement the function manually.
2. Compare it with one prepared candidate.
3. Mark accept, revise, or reject.
4. Run the same tests.
5. Record the same evidence.

# Lab handoff

---

You will evolve one Python utility module:

1. complete `normalize_username` and `is_palindrome`;
2. decide how apostrophes count in `word_frequency`;
3. add `profile_summary`;
4. rename `username` to `handle`;
5. run the full test suite and review the diff.

**Deliverable:** completed code, five passing tests, and a decision record.